

Relatório 2: Operações matriciais em Assembly

João Bruno dos Santos, João Lucas Medina Kormann

Universidade Federal de Santa Catarina

I. INTRODUÇÃO

Este relatório tem o objetivo de apresentar a metodologia aplicada para a resolução dos problemas do Laboratório 2 da disciplina Organização de Computadores I, ministrada pelo professor Marcelo Daniel Berejuck, utilizando a linguagem Assembly para a arquitetura MIPS (Microprocessor without Interlocked Pipelined Stages) e o simulador MARS (MIPS Assembler and Runtime Simulator). O trabalho consiste no desenvolvimento de um programa capaz de realizar operações com matrizes — incluindo a geração da transposta, a multiplicação e o armazenamento dos resultados em memória e em arquivo — seguindo as diretrizes de utilização de registradores.

II. IMPLEMENTAÇÃO DO CÁLCULO MATRICIAL

Inicialmente para a implementação do cálculo matricial, dividimos em três procedimentos principais: Procedimento de matriz transposta (PROC_TRANS), Procedimento de multiplicação matricial (PROC_MUL) e o procedimento de escrita do resultado (MATRIX_TO_FILE e WRITE_CHAR). Como inicialização padrão dos parâmetros, utilizamos os registradores de argumentos \$a0, \$a1 e \$a2 para salvarmos o endereço das matrizes a, b e b transposto (iniciada como nula a princípio), partimos então para PROC_MUL utilizando um jal que salva o “ponto seguro” em \$ra para retornarmos assim que a multiplicação é finalizada, logo após salvamos o resultado de PROC_MUL em \$a0 e utilizamos outro jal para o processo de escrita MATRIX_TO_FILE onde, assim como no caso anterior, retornamos quando finalizado, partindo então para exit com a função j, que carrega o valor 10 em \$v0 e usa o *syscall*, fechando assim o programa.

```
.data
a:  .word 1, 2, 3
    .word 0, 1, 4
    .word 0, 0, 1
b:  .word 1, -2, 5
    .word 0, 1, -4
    .word 0, 0, 1
b_t: .word 0, 0, 0
     .word 0, 0, 0
     .word 0, 0, 0
c:  .word 0, 0, 0
     .word 0, 0, 0
     .word 0, 0, 0
```

Figura 1 - .data matrices.

```
.text
main:
    la    $a0, a
    la    $a1, b
    la    $a2, b_t

    jal   PROC_MUL

    move  $a0, $v0

    jal   MATRIX_TO_FILE

    j     exit
```

Figura 2 - Inicialização main.

A. Procedimento de Matriz Transposta - PROC_TRANS

Partindo para a explicação da transposição da matriz, inicialmente temos as definições de \$t1-\$t4, os registradores temporários que utilizamos para auxiliar nos cálculos e processos deste bloco.

```
PROC_TRANS:
    li    $t1, 0
    li    $t2, 0
    move  $t3, $a0          # $t3 = b
    move  $t4, $a1          # $t4 = b_t
```

Figura 3 - Inicialização PROC_TRANS.

Enquanto \$t1 e \$t2 são iniciados imediatamente como 0 para servirem posteriormente de contadores, \$t3 e \$t4 recebem os valores referentes à b e b transposto que já foram previamente definidos em \$a0 e \$a1.

Em seguida, para o código efetivo, utilizamos uma lógica simples e intuitiva para o cálculo da transposta. Utilizando o princípio dos endereços de memórias e forma de armazenamento dos termos

“words”, carregamos o valor da *word* referente ao endereço de \$t3 (b) em um registrador temporário \$t0 e logo após, com este registrador temporário, sobrepomos este valor no endereço de \$t4. Executado este processo uma vez, somamos +4 a \$t3 para assim olharmos a próxima *word*, enquanto para \$t4, somamos +12, fazendo assim com que o endereço ao qual o valor obtido em \$t3(2º elemento da primeira linha) fosse salvo seria o primeiro elemento da segunda coluna, executando assim a transposição. Além disso, é somado também +1 para \$t1 servindo como um contador, fazendo assim com que este procedimento fosse executado duas vezes, totalizando 3 obtenções de dados e “transposições parciais”, devido ao fato de ser uma matriz 3x3.

```
lw      $t0, 0($t3)
sw      $t0, 0($t4)

addi    $t3, $t3, 4
addi    $t4, $t4, 12
addi    $t1, $t1, 1          # Contador de linhas

ble     $t1, 2, trans_loop   # Reinicia o loop de linhas
```

Figura 4 - Loop inicial transposição.

Passado o primeiro momento, reiniciamos então o contador \$t1 e subtraímos o registrador \$t4 por 32, valor para o qual a nossa referência se tornaria agora a segunda coluna e repetimos o processo mais duas vezes também, obtendo assim a segunda linha de \$t3 no lugar da segunda coluna de \$t4 e, por fim, a terceira linha de \$t3 no lugar da terceira coluna de \$t4.

```
subi    $t4, $t4, 32          # Voltar o offset pra proxima coluna
addi    $t2, $t2, 1          # Contador de colunas

ble     $t2, 2, trans_loop   # Reinicia o loop de colunas
```

Figura 5 - Loop colunas transposição.

Enfim, salvamos o valor da matriz transposta \$t4 no registrador geral \$v0 e utilizamos o comando jr para retornar a parte do código de onde a função PROC_TRANS foi chamada.

```
move    $v0, $t4
jr      $ra
```

Figura 6 - Parte final PROC_TRANS.

B. Procedimento de Multiplicação Matricial - PROC_MUL

Para o procedimento do cálculo da multiplicação entre as matrizes, inicialmente salvamos os valores dos registradores de argumentos \$a0 - \$a2 (matrizes a, b e b_t) nos registradores \$s1-\$s3 e logo após alteramos \$a0 para b e \$a1 para b_t, utilizando então o comando jal para pularmos até a função PROC_TRANS enquanto salvamos o endereço deste passo no registrador \$ra, permitindo assim com que seja retornado, assim que finalizado PROC_TRANS, para ponto de partida porém desta vez com b transposto já calculado pela função chamada. Utilizamos também um addi -4 para diminuir o valor do endereço de \$sp, enquanto salvamos o \$ra (endereço desta função em si) no novo valor do endereço de \$sp, servindo assim como uma pilha de funções.

```
addi    $sp, $sp, -4          # abre espaço na pilha
sw      $ra, 0($sp)          # salva $ra na pilha
move    $s1, $a0              # s1 = a
move    $s2, $a1              # s2 = b
move    $s3, $a2              # s3 = b_t

move    $a0, $s2              # a0 = b
move    $a1, $s3              # a1 = b_t

jal     PROC_TRANS
```

Figura 7 - Passos iniciais PROC_MUL.

Assim que retornado de PROC_TRANS, partimos então para o procedimento em si da multiplicação. De início, zeramos os registradores temporários \$t2-\$t6 apenas como medida de segurança para não haver inconsistências e carregamos \$v0 como c, nossa matriz resultado final.

```
li      $t2, 0
li      $t3, 0
li      $t4, 0
li      $t5, 0
li      $t6, 0

la      $v0, c
```

Figura 8 - Inicialização registradores.

Partindo para o loop inicial, carregamos o valor (*word*) de \$s1(a) e \$s3(b transposto) nos registradores temporários \$t0 e \$t1 e então multiplicamo-nos, salvando seu resultado no registrador \$t2, enquanto somamos os valores de \$t2 em \$t3, utilizando então uma lógica parecida ao caso de PROC_TRANS de soma dos endereços

para interagirmos a linha de \$s1 com a coluna de \$s3, somando também 1 a \$t4 para servir de contador de elementos e repetindo este loop até que \$t4 = 2, caso onde chegamos ao último elemento da linha/coluna em questão. Fazendo algumas iterações como exemplo temos:

$$c_{11} = a_{11} * b_{11} + a_{12} * b_{21} + a_{13} * b_{31}$$

```
lw    $t0, 0($s1)      # carrega primeiro valor de a
lw    $t1, 0($s3)      # carrega primeiro valor de b
mul    $t2, $t1, $t0    # t2 = a*b
add    $t3, $t3, $t2    # incrementa t2 no acc

addi   $s1, $s1, 4      # incrementa a referencia de a para prox coluna
addi   $s3, $s3, 12     # incrementa a referencia de b para prox "linha"
addi   $t4, $t4, 1      # incrementa somador de itens
ble    $t4, 2, loop_item
```

Figura 9 - Calculo item matriz saída.

O valor então de c_{xx} é então salvo no endereço de \$v0, logo após, recomeçamos todo o processo, somando +4 a \$v0 para referenciar o próximo endereço, zerando o acumulador \$t3 para os novos valores, subtraindo -12 de \$s1 para assim retornar ao início da linha, subtraindo -32 de \$s3 para utilizarmos a próxima coluna e adicionando +1 ao contador \$t5, para assim efetuarmos este processo um total de 3 vezes, uma para cada coluna.

```
sw     $t3, 0($v0)      # guarda o valor do acumulador
addi   $v0, $v0, 4      # incrementa a referencia do resultado para a prox palavra
li     $t3, 0           # zera o acumulador

subi   $s1, $s1, 12     # retorna a referencia de a para o inicio
subi   $s3, $s3, 32     # retorna a referencia de b para a segunda coluna
li     $t4, 0           # zera o contador dos itens

addi   $t5, $t5, 1      # soma o contador de iteracoes para cada item
ble    $t5, 2, loop_item
```

Figura 10 - Processo de recomeço loop.

Após executar o cálculo das 3 colunas com a linha referente, passamos então para a próxima linha da matriz a, voltando o contador \$t5 para 0, adicionando +12 a \$s1 para referenciar a próxima linha, subtraindo -12 de \$s3 para retornar à primeira coluna e adicionando +1 ao contador de linhas \$t6 que, assim como o contador de colunas \$t5, vai contar até 2 induzindo o processo a executar para as três linhas que, por sua vez, iteram cada uma com as 3 colunas da outra matriz.

```
li     $t5, 0
addi   $s1, $s1, 12
subi   $s3, $s3, 12
addi   $t6, $t6, 1
ble    $t6, 2, loop_item
```

Figura 11 - Loop próxima linha.

Por fim, retornamos \$v0 para a referência inicial da matriz resultado (c), restauramos o \$ra da pilha

de funções e adicionamos +4 a \$sp para liberar este espaço ocupado da memória, finalizando com um jr para o endereço que restauramos de \$ra.

C. Procedimento de escrita do resultado

De modo a realizar as premissas propostas, foi implementado o procedimento *MATRIX_TO_FILE* que recebe como argumento o endereço do início de uma matriz 3x3 em \$a0, preenchida com inteiros positivos e negativos de até dois dígitos, e exporta os dados para um arquivo .txt. Vale ressaltar que, como esse é um procedimento não-folha, é utilizada a *stack* para guardar o endereço de chamada.

Para o início do procedimento, é necessário abrir um arquivo para então ser escrito, como mostra a figura x. A sequência de instruções foi retirada do código exemplo da documentação, e é responsável por abrir e registrar o identificador do arquivo para uso de escrita e fechá-lo posteriormente.

```
### Abertura de arquivo ###
li     $v0, 13          # system call for open file
la     $a0, filename    # output file name
li     $a1, 1           # Open for writing (flags are 0: read, 1: write)
li     $a2, 0           # mode is ignored
syscall                               # open a file (file descriptor returned in $v0)
move   $s6, $v0         # save the file descriptor
#####
```

Figura 12 - Instruções de abertura de arquivo.

A escrita em si é efetuada por um procedimento folha chamado *WRITE_CHAR*, responsável por efetuar a escrita de um único byte no arquivo já aberto. Como argumento é recebido o endereço do buffer de 1 byte que deverá ser escrito.

```
WRITE_CHAR:
li     $v0, 15          # system call for write to file
move   $a0, $s6         # file descriptor
li     $a2, 1           # hardcoded buffer length
syscall                               # write to file
jr     $ra
```

Figura 13 - Função para a escrita de 1 byte.

Com o arquivo aberto, o algoritmo de escrita é dividido em três partes, uma para tratar números

negativos, a segunda para tratar números com dois dígitos, e a última para tratar o dígito único ou restante. No início do procedimento, o argumento é movido para \$t0, e os registradores \$t5 e \$t6 são definidos como zero para uso de contadores internos.

Em \$t1, é carregado o número inteiro. Como exibido na figura x, a primeira checagem feita no programa, com a instrução *bgez* (*branch if greater or equal zero*), pula para a label *positive* caso o número seja positivo. Caso contrário, passa pelas instruções para imprimir o caractere '-' no arquivo.

```
println:
lw    $t1, 0($t0)          # carregando elemento
bgez  $t1, positive        # se for positivo

sub    $t1, $zero, $t1      # se for negativo vira positivo
la     $a1, minus          # address of buffer from which to write
jal    WRITE_CHAR

positive:
```

Figura 14 - Tratamento de valores negativos.

Após isso, é feita a checagem da quantidade de caracteres do número. Para isso, é utilizada a função *div* (divisão com *overflow*), onde o resultado da divisão fica guardado no registrador LO, e o resto em HI. Com isso, podemos separar o valor das dezenas e das unidades, em \$t3 e \$t4, respectivamente.

A verificação dessa etapa é feita com a instrução *beqz* (*branch if equals zero*), comparando se o valor das dezenas está zerado, ou seja, o número em questão é menor que 10. Caso possua valor nas dezenas, é armazenado no arquivo utilizando o *WRITE_CHAR*, se não, o programa passa para a label *singleDigit*.

Vale ressaltar que, como estamos tratando de valores inteiros, e não caracteres, essa etapa demanda a conversão. Com o valor já separado por *div*, é possível só adicionar 48, o valor de '0' na tabela ASCII (*American Standard Code for Information Interchange*) para realizar a conversão. Assim, é preciso salvar somente o byte respectivo do número utilizando um *charBuffer* criado anteriormente, e passar como argumento para *WRITE_CHAR*.

```
li     $t2, 10
div    $t1, $t2            # divide elemento por 10
mflo   $t3                # quociente
mfhi   $t4                # resto

beqz   $t3, singleDigit    # se tiver so um dígito (quociente = 0)

addi   $t3, $t3, 48
sb     $t3, charBuffer
la     $a1, charBuffer
jal    WRITE_CHAR

singleDigit:
```

Figura 15 - Impressão de dezenas.

Com um único dígito, já tratado previamente, só é necessário realizar a conversão para caractere com o processo citado anteriormente, e registrar no arquivo. A cada iteração do código citado acima, é impresso um espaço em branco. A cada 3 iterações, é impressa uma nova linha. O processo encerra depois de 9 iterações, e é responsável por fechar o arquivo criado nele mesmo.

O nome do arquivo a ser impresso é definido pelo buffer *filename*, que por vez tem seu nome decidido pelo procedimento *PROC_NOME*, que pede a entrada do usuário pelo terminal, e a carrega.

```
PROC_NOME:
# Exibe mensagem pedindo o nome
li     $v0, 4
la     $a0, askName
syscall

# Lê a string digitada
li     $v0, 8
la     $a0, filename
li     $a1, 64
syscall

jr     $ra
```

Figura 16 - Implementação de PROC_NOME

III. RESULTADOS E LIMITAÇÕES

Com o código apresentado acima, é possível perceber o cumprimento das premissas propostas. A função *PROC_TRANS* realiza a transposição de uma matriz 3x3 sendo folha, *PROC_MUL* realiza a multiplicação de matrizes do mesmo tamanho, é não folha e armazena o resultado na memória de dados.

A impressão dos resultados é feita com sucesso, com o nome do arquivo a ser decidido pelo usuário por conta do processo *PROC_NOME*.

A utilização das matrizes entre um processo e outro é feito por meio de argumentos nos

registradores \$a0-\$a3, e os valores resultantes são passados por \$v0-\$v1.

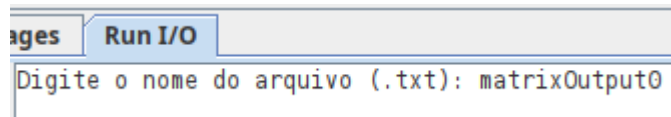


Figura 17 - Entrada do usuário

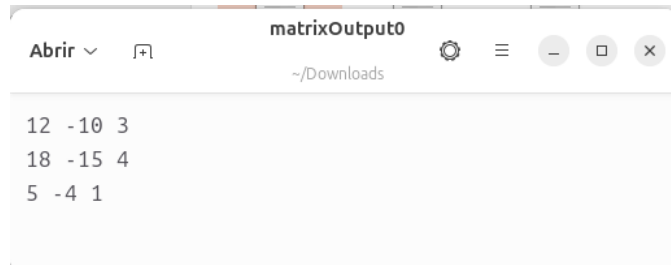


Figura 18 - Arquivo de saída

Escalabilidade é o problema principal da versão entregue, já que suporta a operação exclusivamente de matrizes 3x3 e a impressão dos resultados feita por `MATRIX_TO_FILE` implementa somente valores inteiros de dois dígitos. Contudo, como o exercício propõe entrada limitada de dados já conhecidos, e por via a saída não é alterada, apenas das limitações o resultado esperado é obtido com êxito.